

Programación de servicios y procesos
CASO 1 - UD 4 - 2º DAM
José Cruces Aparicio

- **TÍTULO**

CLIENTE DE CORREOS CON JAVAMAIL

- **SITUACIÓN**

Trabajamos en una empresa que desarrolla software para farmacias para el stock y control de los fármacos en tienda. En el software que tenemos desarrollado, pretendemos incluir un módulo que se pueda comunicar con el correo al servidor SMTP de Gmail usando la cuenta de correo propia o ficticia.

Gmail acepta tanto el protocolo SSL como lo TLS para realizar la autenticación y que JavaMail solo requiere definir adecuadamente las propiedades en la hora de instanciar un objeto Session para indicarle que queremos autenticarnos con uno u otro sistema.

- **INSTRUCCIONES**

Se pretende realizar una clase que:

1. Permita conectarse al servidor SMTP de Gmail usando una cuenta de correo Gmail. Tenéis que saber que Gmail acepta tanto el protocolo SSL como lo TLS para realizar la autenticación y que JavaMail solo requiere definir adecuadamente las propiedades en la hora de instanciar un objeto Session para indicarle que queremos autenticarnos con uno u otro sistema.
2. La clase tiene que disponer también de métodos para añadir los destinatarios e indicar el remitente, el tema del correo y el cuerpo. Usáis la sintaxis siguiente:

- a. `void addRecipients(String[] recipients):` para añadir un grupo de destinatarios.
- b. `void addRecipient(String recipient):` para añadir un destinatario
- c. `void setSender(String psender):` para asignar el remitente
- d. `void setSubject(String subject):` para asignar el tema
- e. `void setMailText(String pbody):` para asignar el cuerpo

El puerto y la dirección del servidor SMTP se asignará en el momento de instanciar la clase, en el constructor. Además la clase tendrá 3 métodos específicos para enviar el mensaje:

- **`void sendUsingSSLAuthentication(final String user, final String pass) throws MessagingException:`** para hacer un envío usando una autenticación SSL
- **`void sendUsingTLSAuthentication(final String user, final String pass) throws MessagingException:`** para hacer un envío usando una autenticación TSL
- **`void send() throws MessagingExceptio:`** para hacer un envío sin autenticación

1. DEFINICIÓN E IDENTIFICACIÓN DEL PROBLEMA

La empresa desarrolla software de gestión para farmacias, incluyendo control de stock y gestión de fármacos. Se desea incorporar un módulo que permita enviar correos electrónicos directamente desde la aplicación, utilizando el servidor SMTP de Gmail.

El sistema debe:

- Conectarse al servidor SMTP de Gmail.
- Permitir autenticación mediante **SSL** o **TLS**.
- Permitir envío con o sin autenticación.
- Gestionar destinatarios, remitente, asunto y cuerpo del mensaje.
- Configurar servidor y puerto en el constructor de la clase.

El reto técnico consiste en configurar correctamente las propiedades de **JavaMail (Jakarta Mail)** para cada tipo de autenticación y encapsular la funcionalidad en una clase reutilizable y estructurada.

2. RESOLUCIÓN DEL PROBLEMA

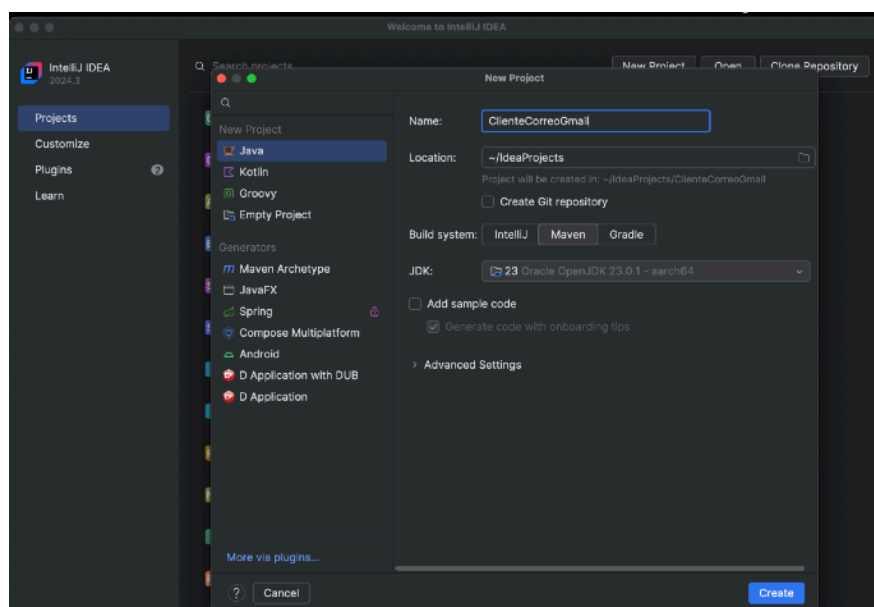
Configuración SMTP de Gmail

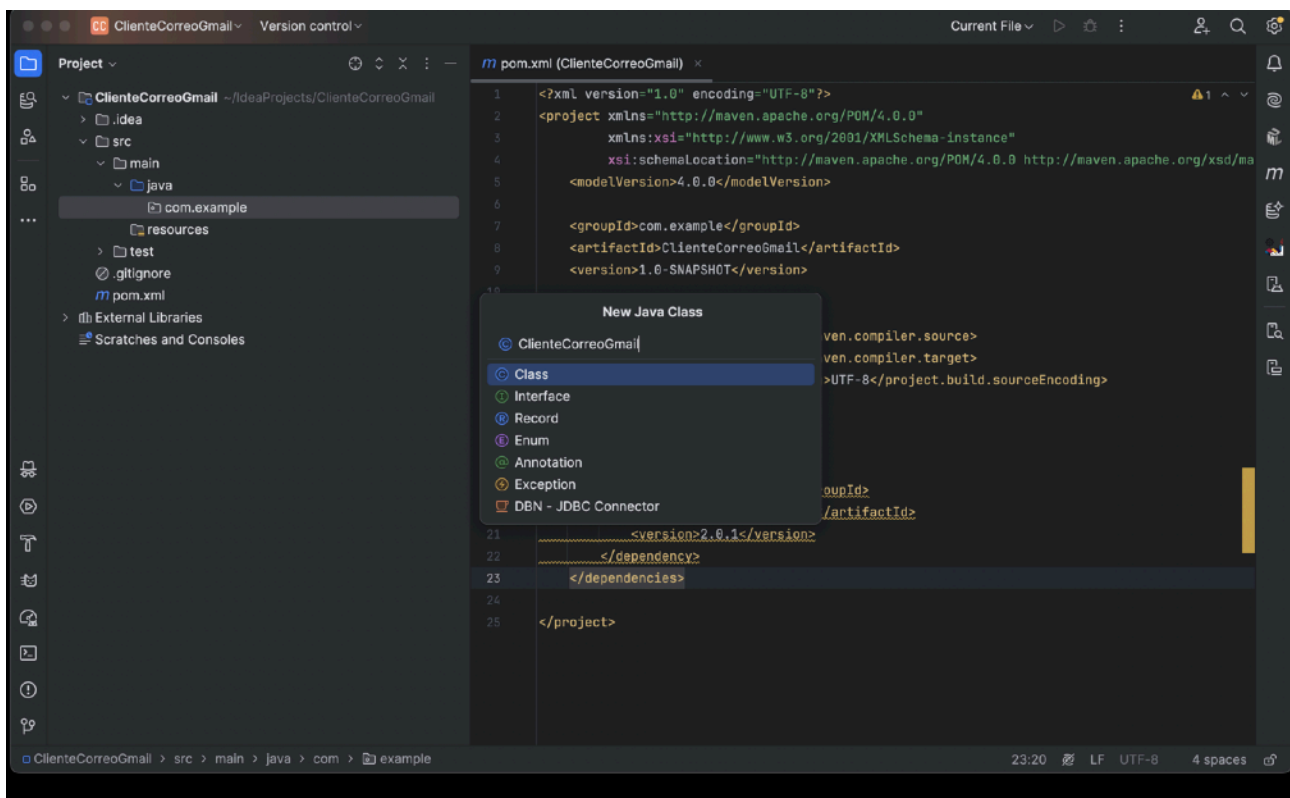
Protocolo --- Servidor SMTP --- Puerto

SSL --- smtp.gmail.com --- 465

TLS --- smtp.gmail.com --- 587

Creacion de proyecto ClienteCorreoGmail en la cual creamos una clase ClienteCorreoGmail y una clase Main para probar y ejecutar el proyecto .





Clase ClienteCorreoGmail

```
package com.example;
```

```
import java.util.Properties;  
import jakarta.mail.*;  
import jakarta.mail.internet.*;
```

```
public class ClienteCorreoGmail {
```

```
    private String smtpHost;  
    private String smtpPort;  
    private Session session;  
    private MimeMessage message;
```

```
    public ClienteCorreoGmail(String host, String port) {  
        this.smtpHost = host;  
        this.smtpPort = port;  
    }
```

```
    // =====  
    // CREAR SESIÓN TLS  
    // =====
```

```
    public void sendUsingTLSAuthentication(final String user, final String pass) {
```

```
        Properties props = new Properties();  
        props.put("mail.smtp.host", smtpHost);  
        props.put("mail.smtp.port", smtpPort);  
        props.put("mail.smtp.auth", "true");  
        props.put("mail.smtp.starttls.enable", "true");
```

```
        session = Session.getInstance(props,  
            new Authenticator() {  
                protected PasswordAuthentication getPasswordAuthentication() {  
                    return new PasswordAuthentication(user, pass);  
                }  
            })
```

```

    }
    });

    message = new MimeMessage(session);
}

// =====
// CREAR SESIÓN SSL
// =====
public void sendUsingSSLAuthentication(final String user, final String pass)
    throws MessagingException {

    Properties props = new Properties();

    props.put("mail.smtp.host", smtpHost);
    props.put("mail.smtp.port", smtpPort);
    props.put("mail.smtp.auth", "true");
    props.put("mail.smtp.socketFactory.port", smtpPort);
    props.put("mail.smtp.socketFactory.class",
        "javax.net.ssl.SSLSocketFactory");

    session = Session.getInstance(props,
        new Authenticator() {
            protected PasswordAuthentication getPasswordAuthentication() {
                return new PasswordAuthentication(user, pass);
            }
        });

    message = new MimeMessage(session);
}

// =====
// ENVIAR CORREO
// =====
public void send() throws MessagingException {

    if (message == null) {
        throw new MessagingException("La sesión de correo no está creada.");
    }

    Transport.send(message);
    System.out.println("Correo enviado correctamente.");
}

// =====
// CONFIGURACIÓN DEL MENSAJE
// =====
public void setSender(String sender) throws MessagingException {
    message.setFrom(new InternetAddress(sender));
}

public void addRecipient(String recipient) throws MessagingException {
    message.addRecipient(Message.RecipientType.TO,
        new InternetAddress(recipient));
}

public void addRecipients(String[] recipients) throws MessagingException {
    for (String r : recipients) {
        addRecipient(r);
    }
}

```

```

    }

    public void setSubject(String subject) throws MessagingException {
        message.setSubject(subject);
    }

    public void setMailText(String body) throws MessagingException {
        message.setText(body);
    }
}

```

Main.java

```
package com.example;
```

```

public class Main {

    public static void main(String[] args) {

        try {

            ClienteCorreoGmail cliente =
                new ClienteCorreoGmail("smtp.gmail.com", "587");

            // Crear sesión con autenticación TLS
            cliente.sendUsingTLSAuthentication(
                "josechucruces@gmail.com",
                "CONTRASEÑA_DE_APLICACION"
            );

            // Configurar correo
            cliente.setSender("josechucruces@gmail.com");
            cliente.addRecipient("tamihevia@gmail.com");
            cliente.setSubject("Correo de prueba desde Java");
            cliente.setMailText("Hola, este correo se envió usando Jakarta Mail.");

            // Enviar correo
            cliente.send();

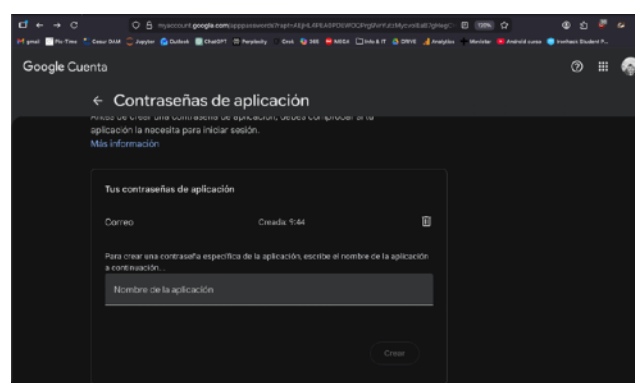
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Para la prueba tenemos que crear en Gmail una contraseña para evitar el doble paso de validación de gmail .

Primer paso ir a google a crear la contraseña de aplicación

<https://myaccount.google.com/apppasswords>



3. ESTRUCTURA Y FUNCIONAMIENTO

Constructor

Permite configurar el servidor SMTP y el puerto.

Gestión de destinatarios

- `addRecipient()`
- `addRecipients()`

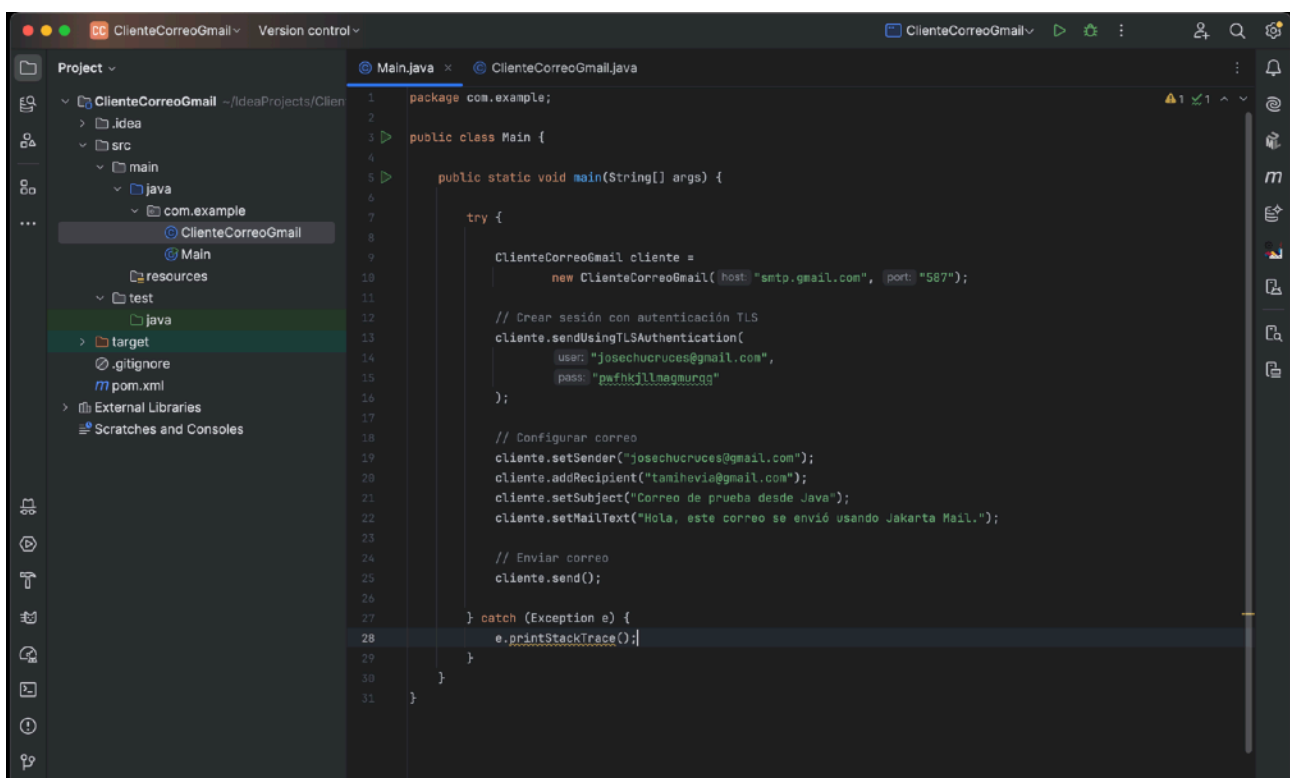
Configuración del mensaje

- `setSender()`
- `setSubject()`
- `setMailText()`

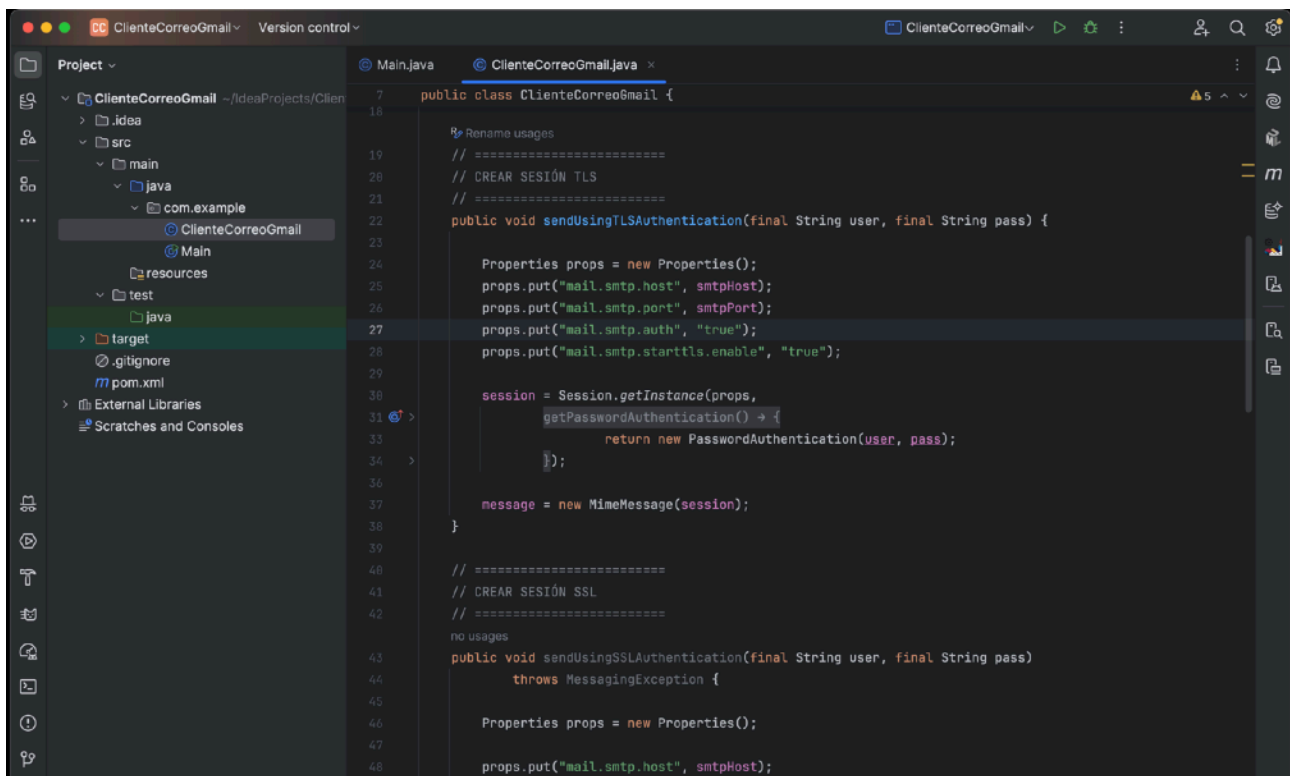
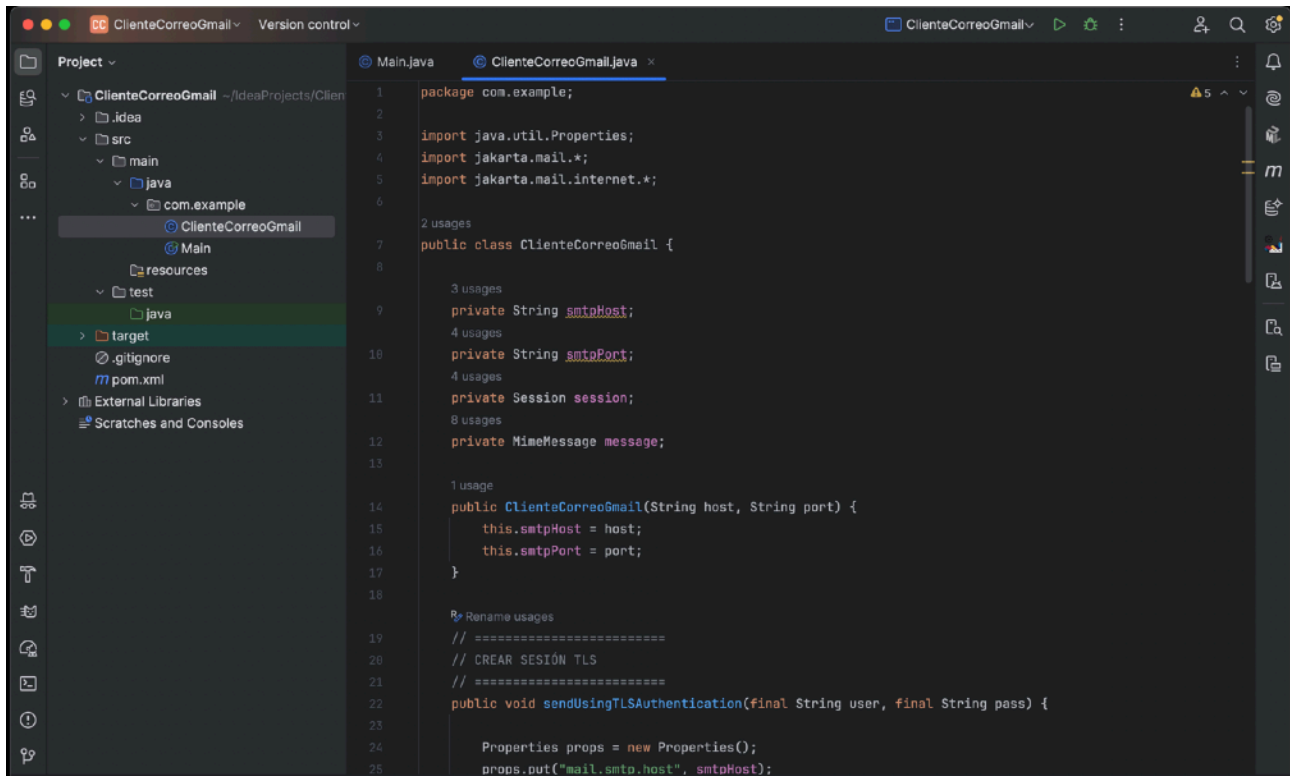
Métodos de envío

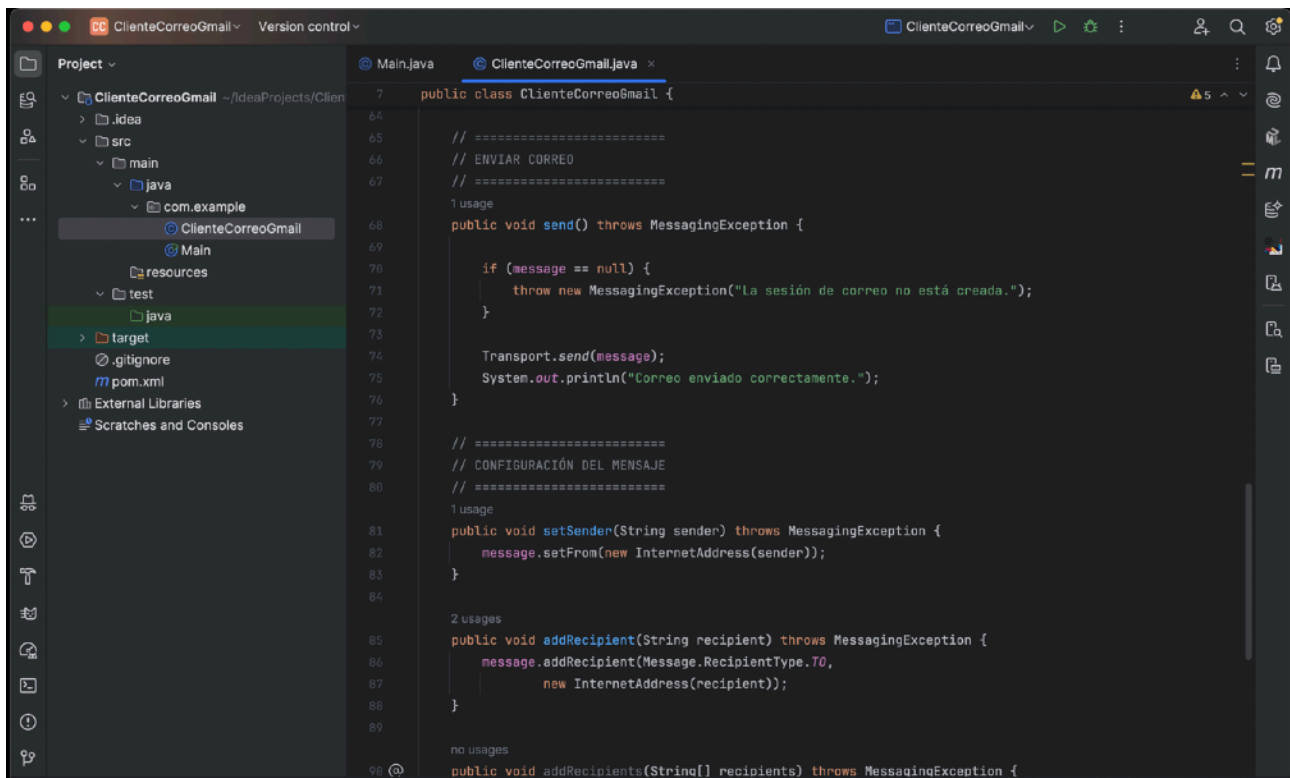
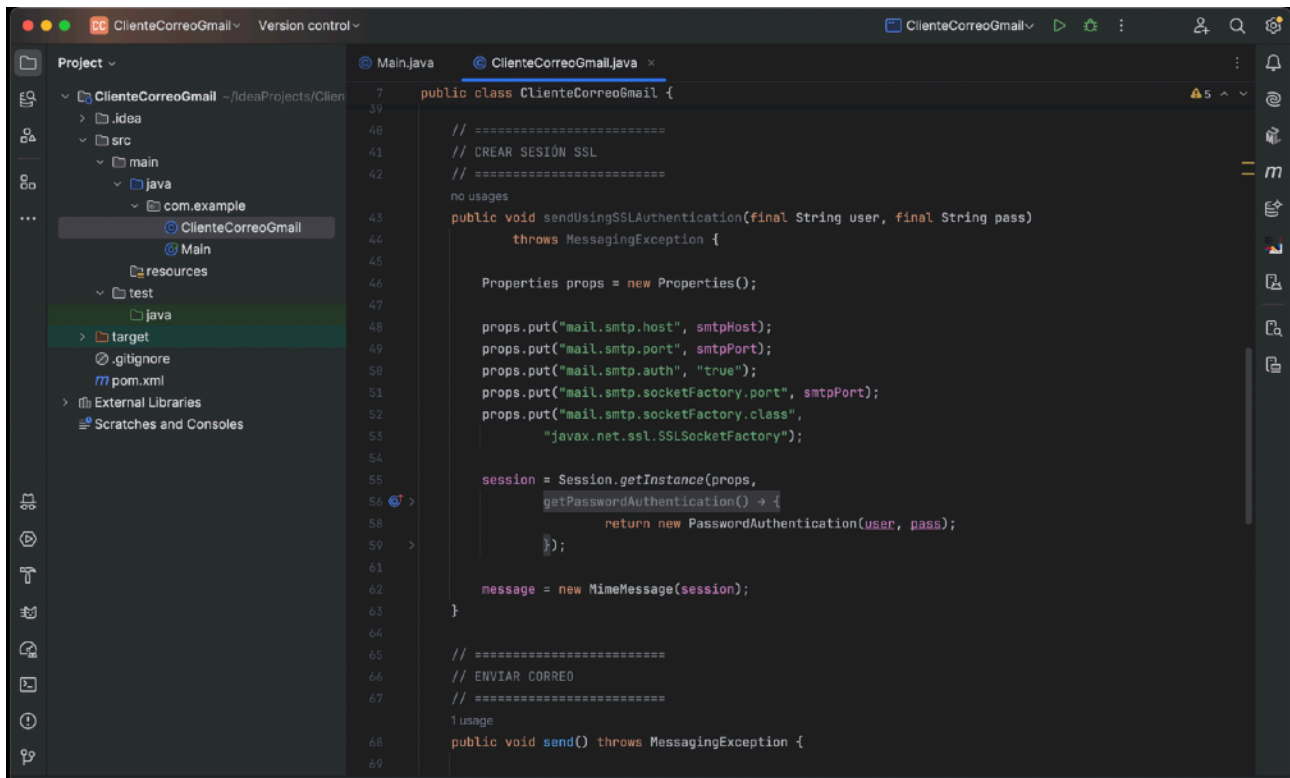
- `send()` → sin autenticación.
- `sendUsingSSLAuthentication()` → puerto 465.
- `sendUsingTLSAuthentication()` → puerto 587.

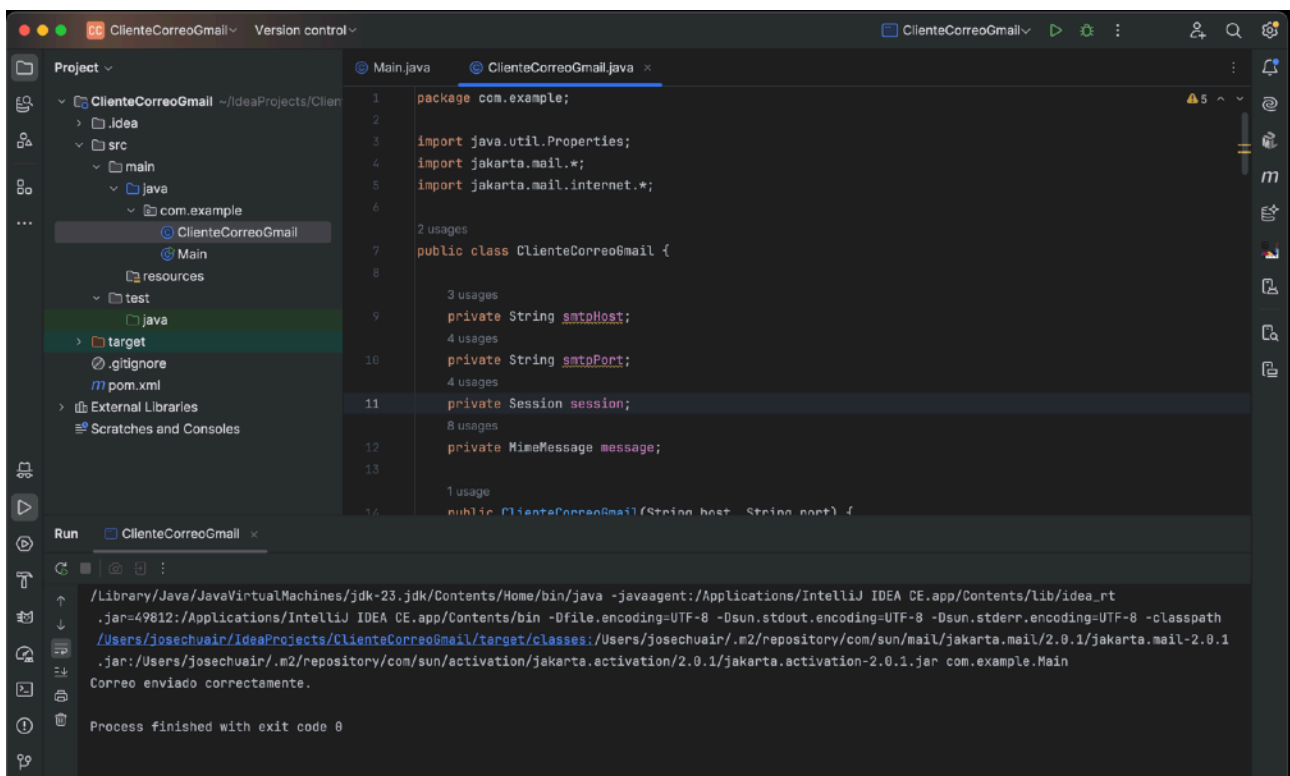
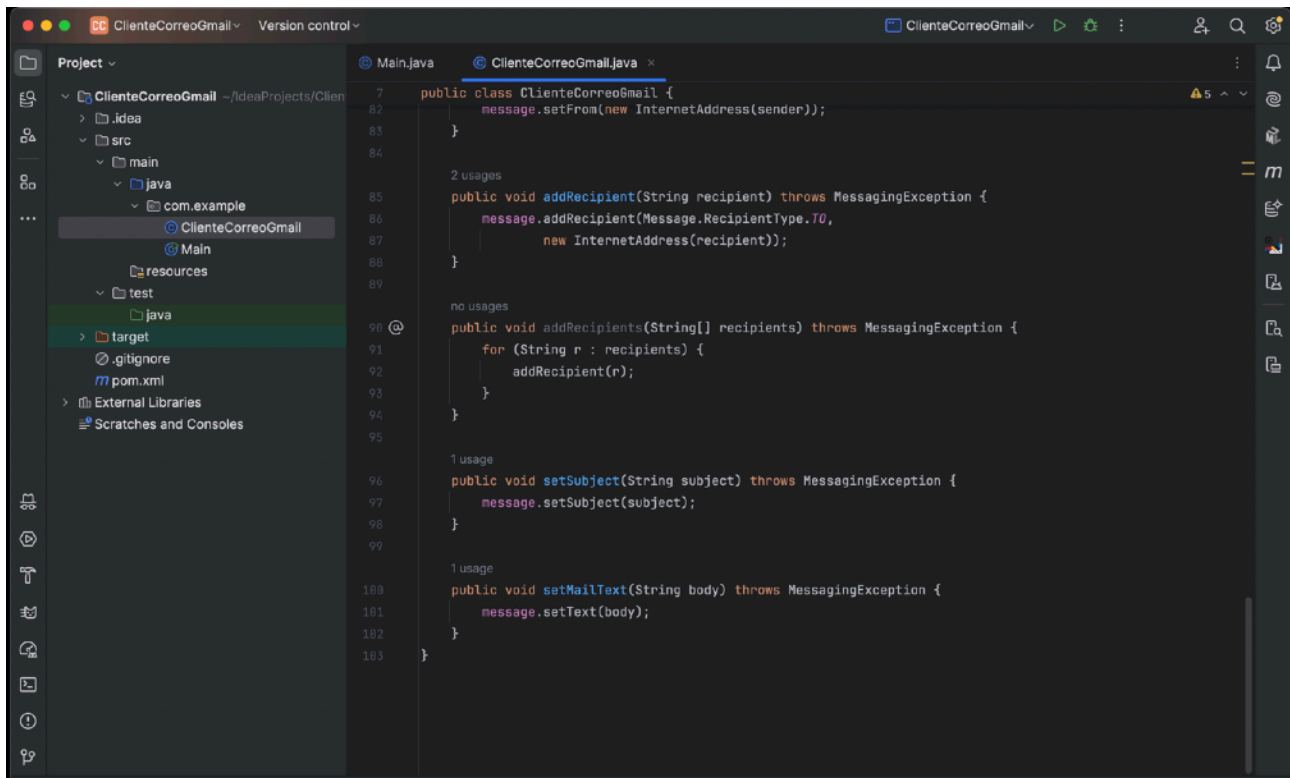
Mostramos aquí que el código de las dos clases y luego la ejecución del main :



```
1 package com.example;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         try {
8
9             ClienteCorreoGmail cliente =
10                 new ClienteCorreoGmail("smtp.gmail.com", "587");
11
12             // Crear sesión con autenticación TLS
13             cliente.sendUsingTLSAuthentication(
14                 "josechucruces@gmail.com",
15                 "pafhkjllnagaurqa");
16         };
17
18         // Configurar correo
19         cliente.setSender("josechucruces@gmail.com");
20         cliente.addRecipient("tamihevia@gmail.com");
21         cliente.setSubject("Correo de prueba desde Java");
22         cliente.setMailText("Hola, este correo se envió usando Jakarta Mail.");
23
24         // Enviar correo
25         cliente.send();
26
27     } catch (Exception e) {
28         e.printStackTrace();
29     }
30 }
31 }
```



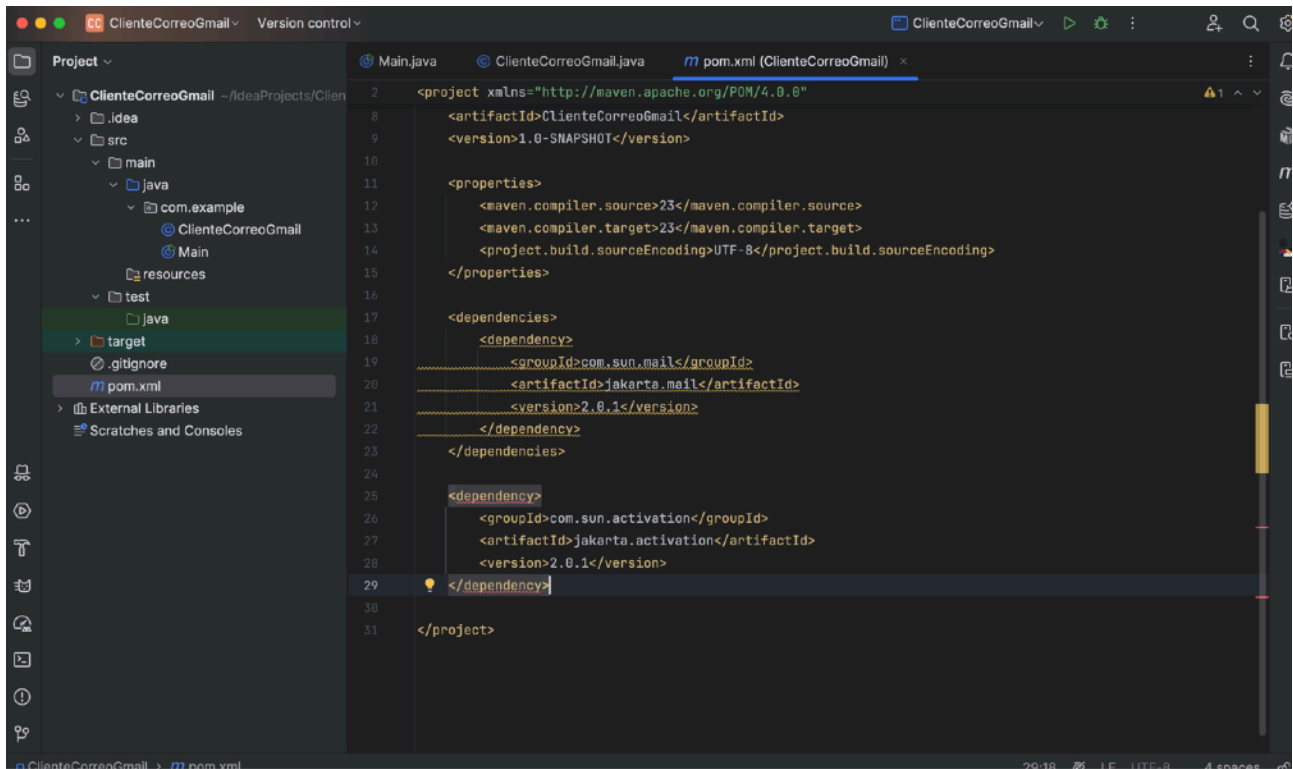




Link a todo el proyecto en drive :

https://drive.google.com/drive/folders/1Da0XSy_RrD_9tlv0nTrGOQWooKe8J5gK?usp=sharing

Y capturas de pantalla de las dependencias incluidas en el pom.xml



4. CONCLUSIONES Y REFLEXIÓN

La implementación del módulo de correo mejora significativamente el software farmacéutico, permitiendo:

- Envío de avisos automáticos de stock.
- Comunicación con proveedores.
- Envío de informes.
- Alertas de caducidad de medicamentos.

El uso de JavaMail permite una integración robusta y flexible, siendo compatible con SSL y TLS mediante la correcta configuración de propiedades.

Desde el punto de vista profesional, encapsular la lógica SMTP en una clase específica mejora:

- Mantenibilidad.
- Reutilización.
- Escalabilidad.
- Separación de responsabilidades.

BIBLIOGRAFIA

Oracle. *JavaMail API Documentation*.
<https://javaee.github.io/javamail/>

Material de la **Unidad Didáctica 4 – Sistemas y Procesos**